

Exercices ROS 2 – URDF XACRO

1. Introduction

Le but de l'exercice est de voir l'utilisation de XACRO et du langage URDF pour rempla

Il comprend :

- L'utilisation de RVIZ et du Robot_State_Publisher pour voir le robot sur une interface graphique et de le manipuler axe par axe.
- L'intégration d'un robot UR5e à la place de l'UR10e dans une scène préconstruite.

2. L'URDF de la scène de base

L'URDF de la scène du workshop intègre :

- Un bras UR10e
- Un bloc « Camera » en bout de bras
- Une plateforme « cube » qui a été placée pour surélever le bras

Copiez le package qui contient l'URDF dans le workspace. Compilez et sourcez :

```
$ cp -r ~/workshop_rosconfr_2025/workshop_robot_description ~/my_ros_ws/src
$ cd ~/my_ros_ws
$ rosdep install --from-paths src -y -r --ignore-src
$ colcon build
$ source install/setup.bash
```

Allez ouvrir le fichier URDF de la plateforme :

```
$ cd ~/my_ros_ws/src/workshop_robot_description/urdf/
$ code . # ouvre l'IDE dans le dossier actuel (remplacez code par votre IDE)
```

Ouvrez le fichier ur10e_with_cube.urdf.xacro

Le contenu de l'URDF est le suivant :

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="workshop">
  <link name="world"/>

  <link name="cube_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.5 0.5 0.5"/>
      </geometry>
      <material name="gray">
        <color rgba="0.6 0.6 0.6 1"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.5 0.5 0.5"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="2.0"/>
      <inertia ixx="0.02" ixy="0.0" ixz="0.0"
        iyy="0.02" iyz="0.0" izz="0.02"/>
    </inertial>
  </link>

  <joint name="world_to_cube" type="fixed">
    <parent link="world"/>
    <child link="cube_link"/>
    <origin xyz="0 0 0.25" rpy="0 0 0"/>
  </joint>

  <!-- Alternative cube link for an elevator -->
  <!--
  <joint name="world_to_cube" type="prismatic">
    <parent link="world"/>
```

Formation ROS

```
<child link="cube_link"/>
<origin xyz="0 0 0.25" rpy="0 0 0"/>
<axis xyz="0 0 1"/>
<limit lower="0.0" upper="3.0" effort="1000.0" velocity="0.5"/>
</joint>
-->

<xacro:include filename="$(find ur_description)/urdf/ur_macro.xacro"/>
<xacro:include filename="$(find workshop_robot_description)/urdf/ur_ros2_control.xacro"/>

<xacro:arg name="ur_type" default="ur10e"/>
<xacro:arg name="tf_prefix" default=""/>
<xacro:arg name="joint_limit_params" default="$(find ur_description)/config/${arg ur_type}/joint_limits.yaml"/>
<xacro:arg name="kinematics_params" default="$(find ur_description)/config/${arg ur_type}/default_kinematics.yaml"/>
<xacro:arg name="physical_params" default="$(find ur_description)/config/${arg ur_type}/physical_parameters.yaml"/>
<xacro:arg name="visual_params" default="$(find ur_description)/config/${arg ur_type}/visual_parameters.yaml"/>
<xacro:arg name="transmission_hw_interface" default=""/>
<xacro:arg name="safety_limits" default="false"/>
<xacro:arg name="safety_pos_margin" default="0.15"/>
<xacro:arg name="safety_k_position" default="20"/>
<xacro:arg name="force_abs_paths" default="false"/>

<xacro:ur_robot
  name="$(arg ur_type)"
  tf_prefix="$(arg tf_prefix)"
  parent="cube_link"
  joint_limits_parameters_file="$(arg joint_limit_params)"
  kinematics_parameters_file="$(arg kinematics_params)"
  physical_parameters_file="$(arg physical_params)"
  visual_parameters_file="$(arg visual_params)"
  safety_limits="$(arg safety_limits)"
  safety_pos_margin="$(arg safety_pos_margin)"
  safety_k_position="$(arg safety_k_position)"
  force_abs_paths="$(arg force_abs_paths)">
  <origin xyz="0 0 0.25" rpy="0 0 0"/>
</xacro:ur_robot>

<link name="camera_link">
<visual>
  <origin xyz="0.05 0 0.06" rpy="0 0 0"/>
  <geometry>
    <box size="0.06 0.03 0.04"/>
  </geometry>
  <material name="black">
    <color rgba="0.05 0.05 0.05 1"/>
  </material>
</visual>
<collision>
  <origin xyz="0.05 0 0.06" rpy="0 0 0"/>
  <geometry>
    <box size="0.06 0.03 0.04"/>
  </geometry>
</collision>
</link>

<joint name="ee_to_camera" type="fixed">
<parent link="tool0"/>
<child link="camera_link"/>
<origin xyz="0.05 0 0.06" rpy="0 0 0"/>
</joint>

<xacro:ur_ros2_control
  name="$(arg ur_type)"
  tf_prefix="$(arg tf_prefix)"
  kinematics_parameters_file="$(arg kinematics_params)"
  transmission_hw_interface="$(arg transmission_hw_interface)">
</robot>
```

Les différents éléments :

```
<link name="world"/>
```

Cette ligne rajoute un link nommé world. Ce link servira de référence aux autres links. C'est un peu notre origine.

On intègre le support pour le bras comme suit :

```
<link name="cube_link">
<visual>
<origin xyz="0 0 0" rpy="0 0 0"/>
<geometry>
<box size="0.5 0.5 0.5"/>
</geometry>
<material name="gray">
```

Formation ROS

```
<color rgba="0.6 0.6 0.6 1"/>
</material>
</visual>
<collision>
<origin xyz="0 0 0" rpy="0 0 0"/>
<geometry>
<box size="0.5 0.5 0.5"/>
</geometry>
</collision>
<inertial>
<mass value="2.0"/>
<inertia ixx="0.02" ixy="0.0" ixz="0.0" iyy="0.02" iyz="0.0" izz="0.02"/>
</inertial>
</link>
```

On crée un link auquel on ajoute des éléments visuels et de collision. Ici, c'est une boîte (un élément géométrique simple) de 50 cm de côté. On y ajoute également une couleur pour le visuel. L'inertie sert uniquement aux simulations dynamiques (ce que l'on ne fera pas dans ce workshop).

On ajoute un joint (lien de référence) entre le world et le cube_link.

```
<joint name="world_to_cube" type="fixed">
<parent link="world"/>
<child link="cube_link"/>
<origin xyz="0 0 0.25" rpy="0 0 0"/>
</joint>
```

Formation ROS

Ce lien est fixe et est placé à 25cm en hauteur du centre de la scène.

On ajoute ensuite le robot :

```
<xacro:include filename="$(find ur_description)/urdf/ur_macro.xacro"/>
<xacro:include filename="$(find workshop_robot_description)/urdf/ur.ros2_control.xacro"/>

<xacro:arg name="ur_type" default="ur10e"/>
<xacro:arg name="tf_prefix" default="" />
<xacro:arg name="joint_limit_params" default="$(find ur_description)/config/${arg ur_type}/joint_limits.yaml"/>
<xacro:arg name="kinematics_params" default="$(find ur_description)/config/${arg ur_type}/default_kinematics.yaml"/>
<xacro:arg name="physical_params" default="$(find ur_description)/config/${arg ur_type}/physical_parameters.yaml"/>
<xacro:arg name="visual_params" default="$(find ur_description)/config/${arg ur_type}/visual_parameters.yaml"/>
<xacro:arg name="transmission_hw_interface" default="" />
<xacro:arg name="safety_limits" default="false"/>
<xacro:arg name="safety_pos_margin" default="0.15"/>
<xacro:arg name="safety_k_position" default="20"/>
<xacro:arg name="force_abs_paths" default="false"/>

<xacro:ur_robot
  name="$(arg ur_type)"
  tf_prefix="$(arg tf_prefix)"
  parent="cube_link"
  joint_limits_parameters_file="$(arg joint_limit_params)"
  kinematics_parameters_file="$(arg kinematics_params)"
  physical_parameters_file="$(arg physical_params)"
  visual_parameters_file="$(arg visual_params)"
  safety_limits="$(arg safety_limits)"
  safety_pos_margin="$(arg safety_pos_margin)"
  safety_k_position="$(arg safety_k_position)"
  force_abs_paths="$(arg force_abs_paths)">
  <origin xyz="0 0 0.25" rpy="0 0 0"/>
</xacro:ur_robot>
```

Cette partie sera un peu plus développée dans la correction, mais notez les éléments suivants :

```
<xacro:include filename="$(find ur_description)/urdf/ur_macro.xacro"/>
<xacro:include filename="$(find workshop_robot_description)/urdf/ur.ros2_control.xacro" />
```

On intègre deux fichiers externes permettant de charger des macros XACRO, notamment `ur_macro`, qui contient les descriptions des robots UR.

```
<xacro:arg name="ur_type" default="ur10e"/>
```

Cet argument permet de choisir le modèle UR à charger (par exemple `ur10e` ou `ur5e`).

Formation ROS

```
parent="cube_link"
```

Permet de définir que la base de l'UR sera reliée à cube_link.

```
<origin xyz="0 0 0.25" rpy="0 0 0" />
```

Permet de positionner l'UR à 25 cm en hauteur sur le cube (donc à 50 cm de l'origine).

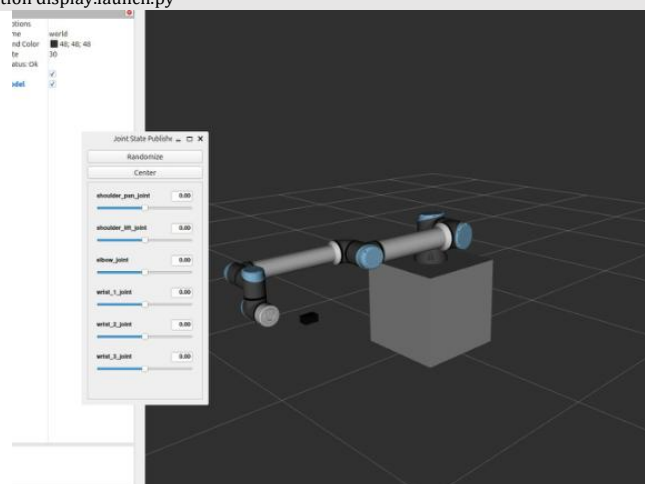
```
<link name="camera_link">
<visual>
  <origin xyz="0.05 0 0.06" rpy="0 0 0"/>
  <geometry>
    <box size="0.06 0.03 0.04"/>
  </geometry>
  <material name="black">
    <color rgba="0.05 0.05 0.05 1"/>
  </material>
</visual>
<collision>
  <origin xyz="0.05 0 0.06" rpy="0 0 0"/>
  <geometry>
    <box size="0.06 0.03 0.04"/>
  </geometry>
</collision>
</link>

<joint name="ee_to_camera" type="fixed">
  <parent link="tool0"/>
  <child link="camera_link" />
  <origin xyz="0.05 0 0.06" rpy="0 0 0"/>
</joint>
```

On ajoute un bloc caméra (comme le cube) à la suite de l'UR (le dernier link de l'UR étant tool0)

Pour visualiser l'URDF, voici la commande :

```
$ ros2 launch workshop_robot_description display.launch.py
```



Essayez de déplacer les axes à l'aide de l'interface graphique.

Formation ROS

Nous allons maintenant chercher à connaître la position du TCP0 dans le monde. Dans un autre terminal:

```
$ ros2 run tf2_ros tf2_echo world tool0 # position de tool0 (TCP0) dans le repère world
```

Déplacez les différents axes pour voir la position évoluer.

Pour continuer :

- Essayez un autre type d'UR (UR5e, UR30, etc.).
- Essayez d'ajouter un autre bloc après tool0 (tout en gardant la caméra). Ajoutez-y des collisions. (Pour d'autres formes géométriques : <https://wiki.ros.org/urdf/XML/link>)
- Modifiez la liaison fixe entre le cube et la base robot par une autre liaison (exemple dans l'URDF déjà présent), mais d'autres possibilités existent : <https://wiki.ros.org/urdf/XML/joint>